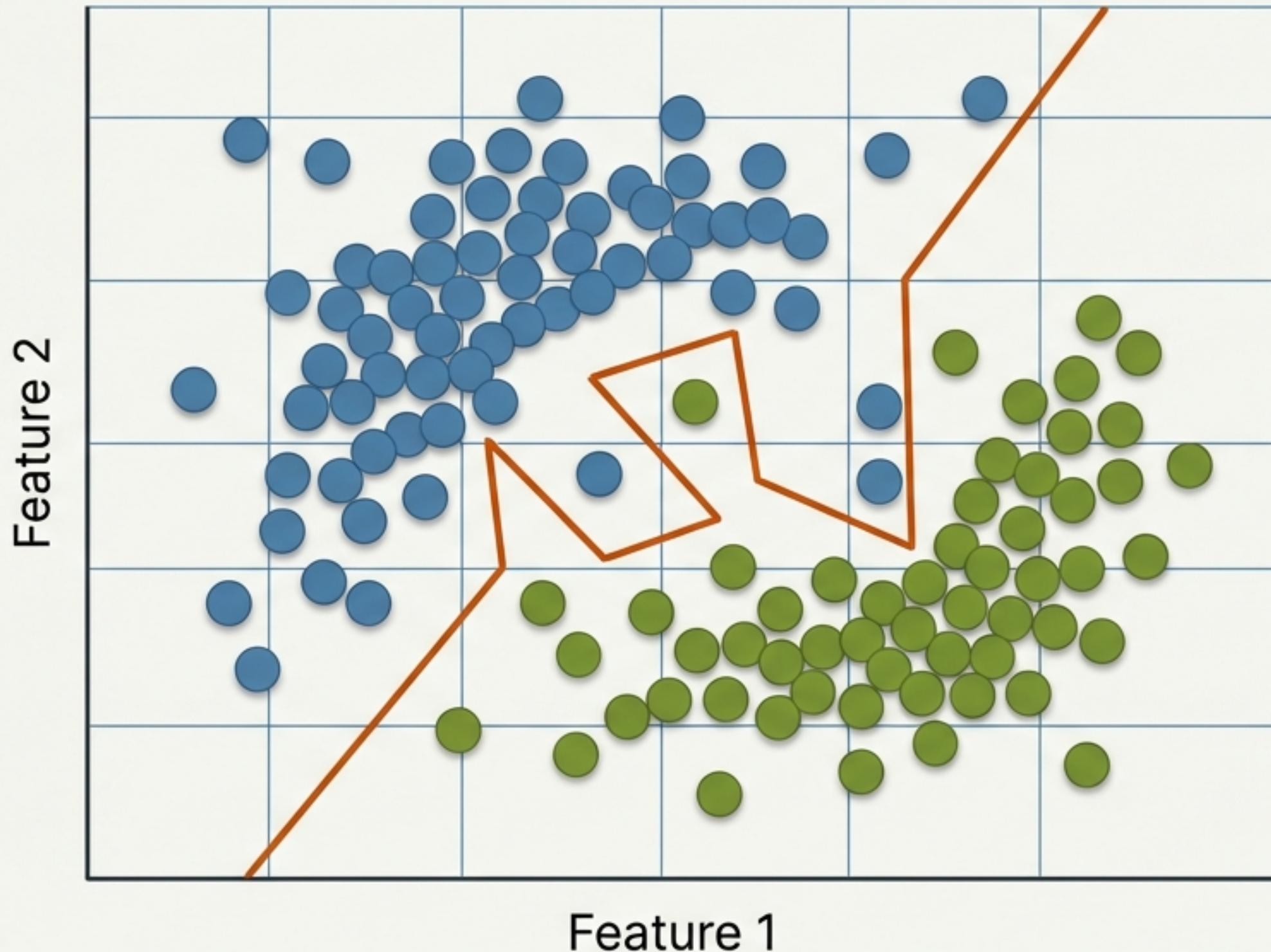


The Bagging Classifiers Playbook



The trap of the single Decision Tree

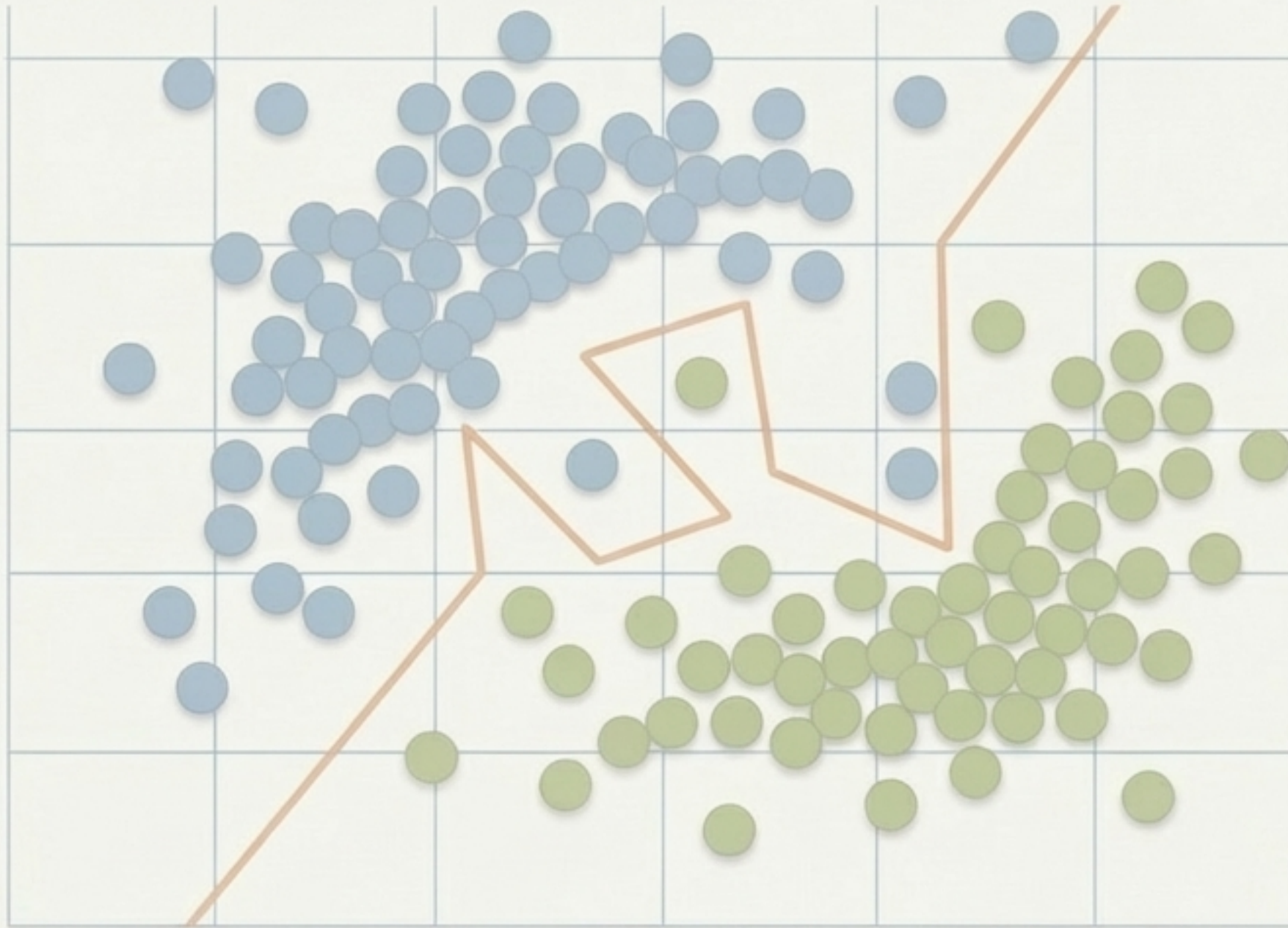


High Variance

The model learns the noise, resulting in poor generalisation on unseen data.

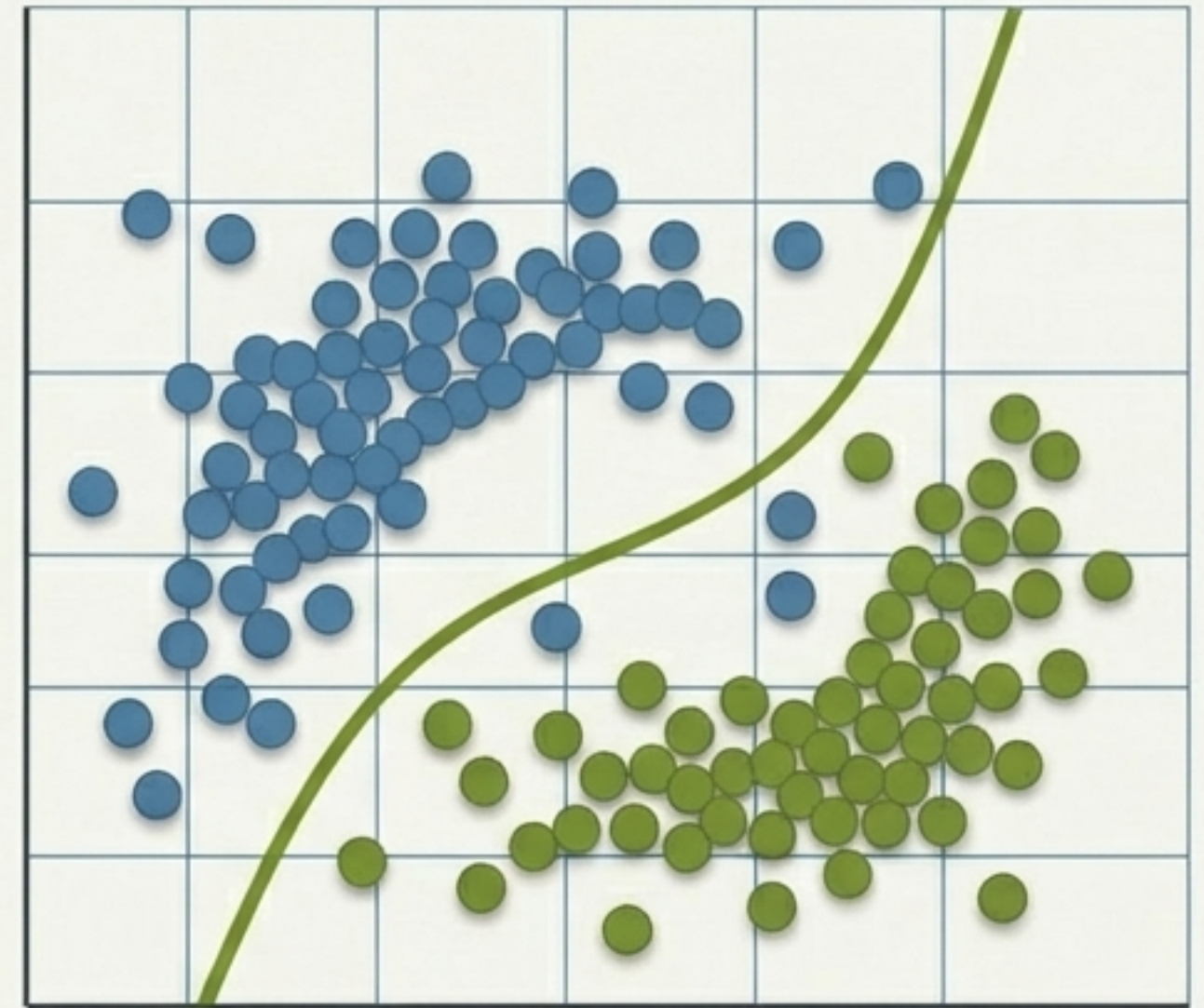
Accuracy tops out around 86% with significant overfitting.

Smoothing the boundary with Bagging



Low Variance

By combining the predictions of multiple models, the ensemble smooths out the individual errors.



Algorithm choice dictates ensemble success

- Decision Trees are ideal base estimators because they naturally have high variance, which bagging efficiently neutralises.
- Support Vector Machines (SVMs) often degrade ensemble performance.

```
base_estimator=DecisionTreeClassifier()
```

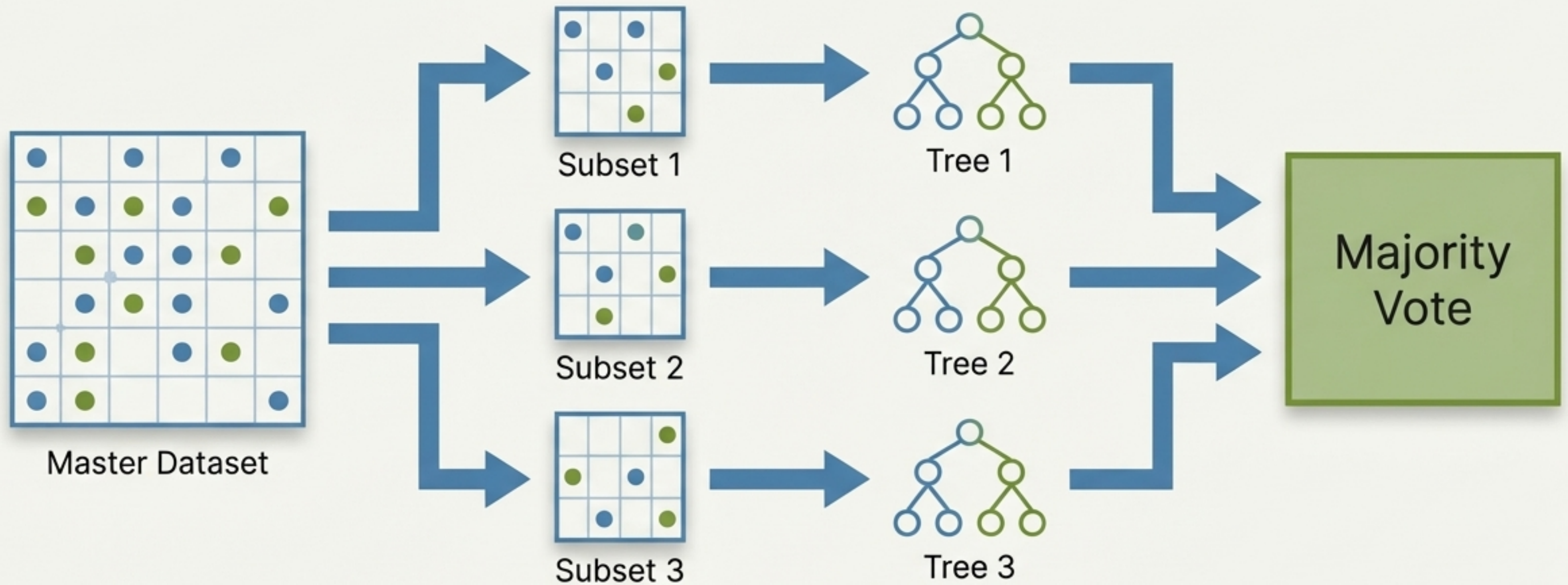
Decision Trees



SVMs

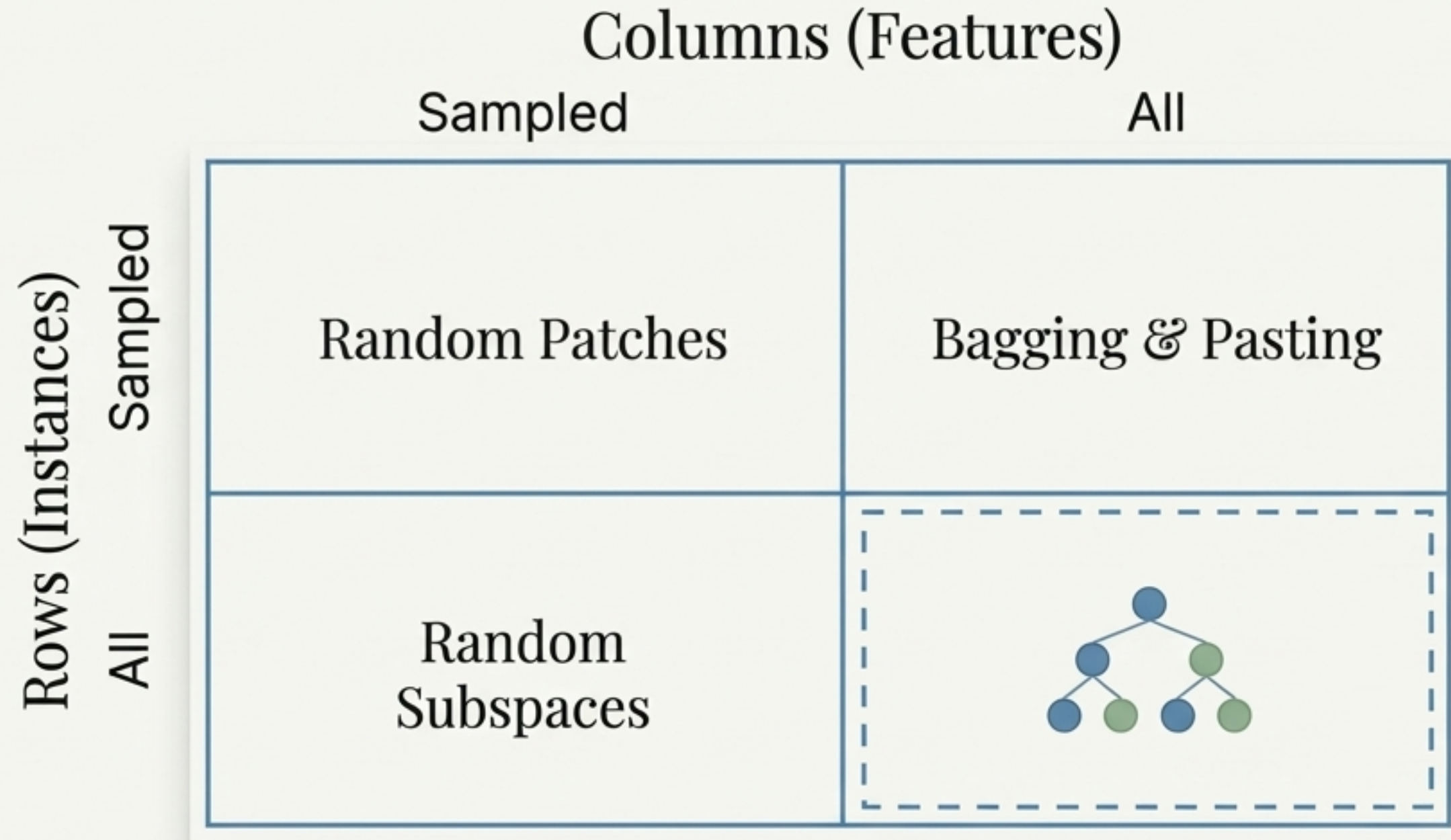


Divide and conquer



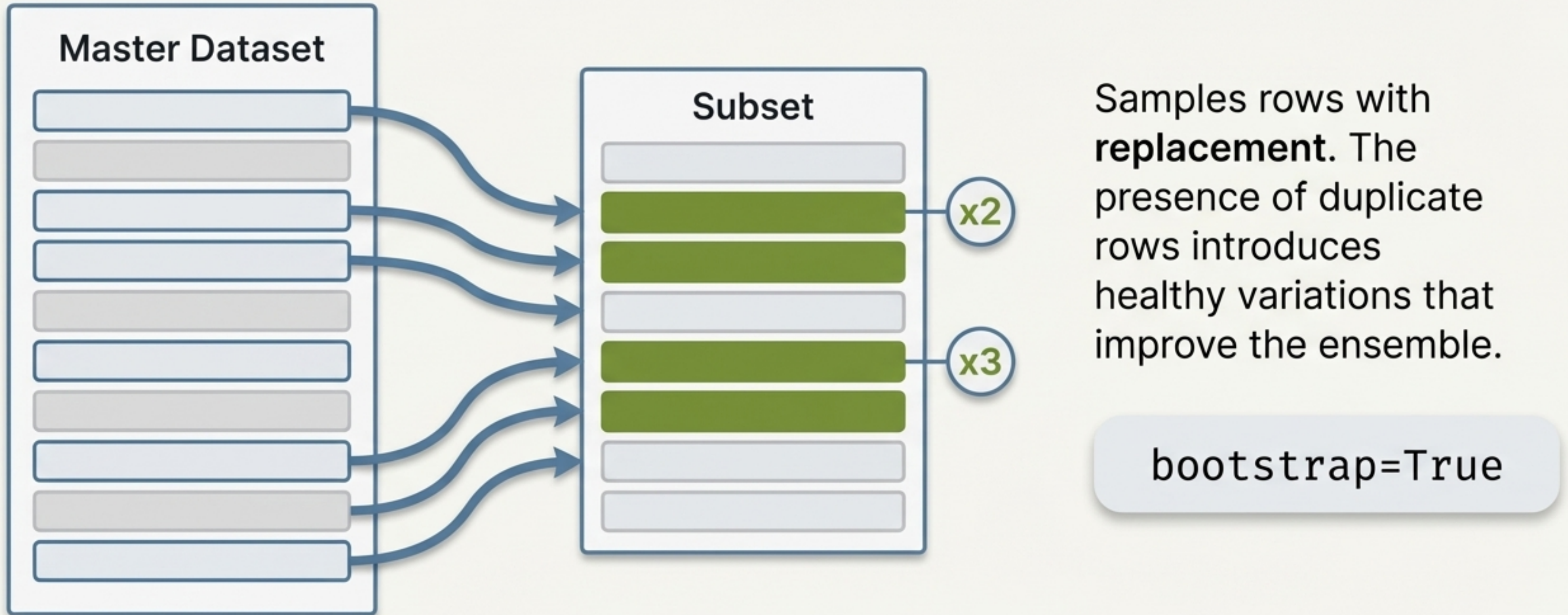
Every base model receives a different subset of the data, ensuring they learn slightly different patterns and make independent errors.

The taxonomy of Scikit-Learn ensembles

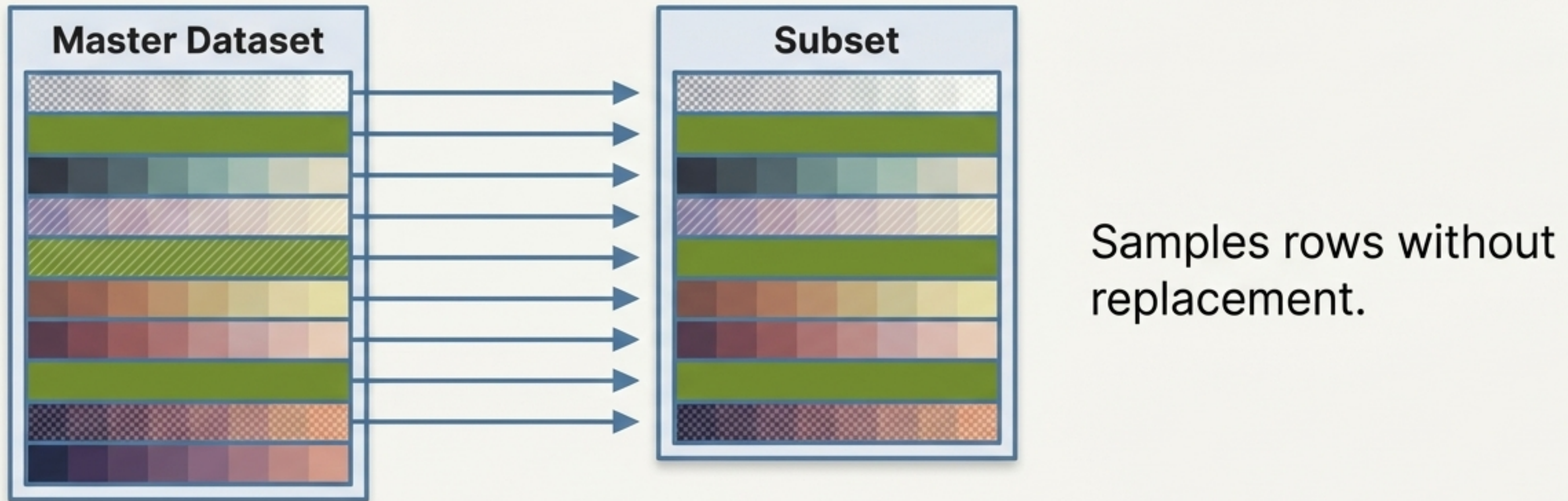


Manipulating Scikit-Learn parameters dictates which quadrant the model occupies.

Method 1: Bagging (Bootstrap Aggregation)



Method 2: Pasting

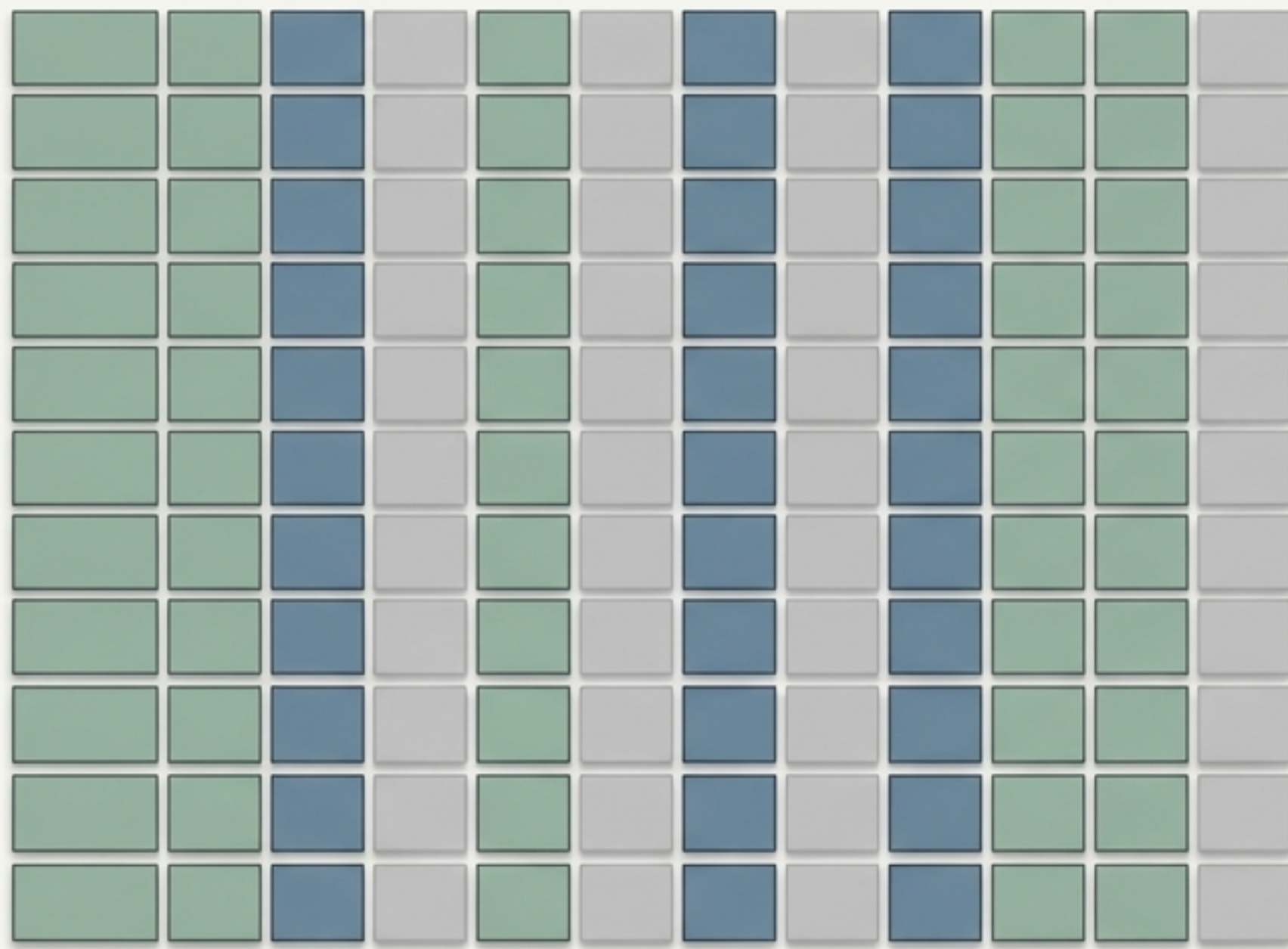


`bootstrap=False`

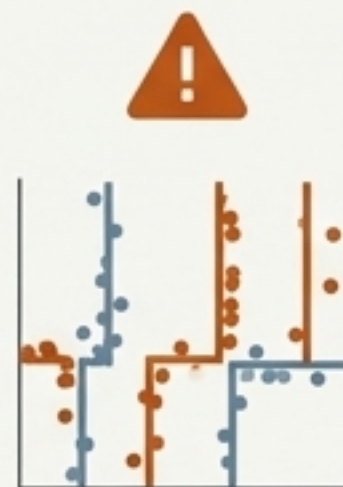
`n_jobs=-1`

Faster to compute (easily split across processor cores), but typically yields **slightly lower accuracy** because it lacks the **variation** introduced by **bagging**.

Method 3: Random Subspaces



Master Dataset



Uses all rows, but samples only a subset of features.

Strictly for high-dimensional datasets (e.g., text, genomics). Applying this to a dataset with only two features will destroy model performance.

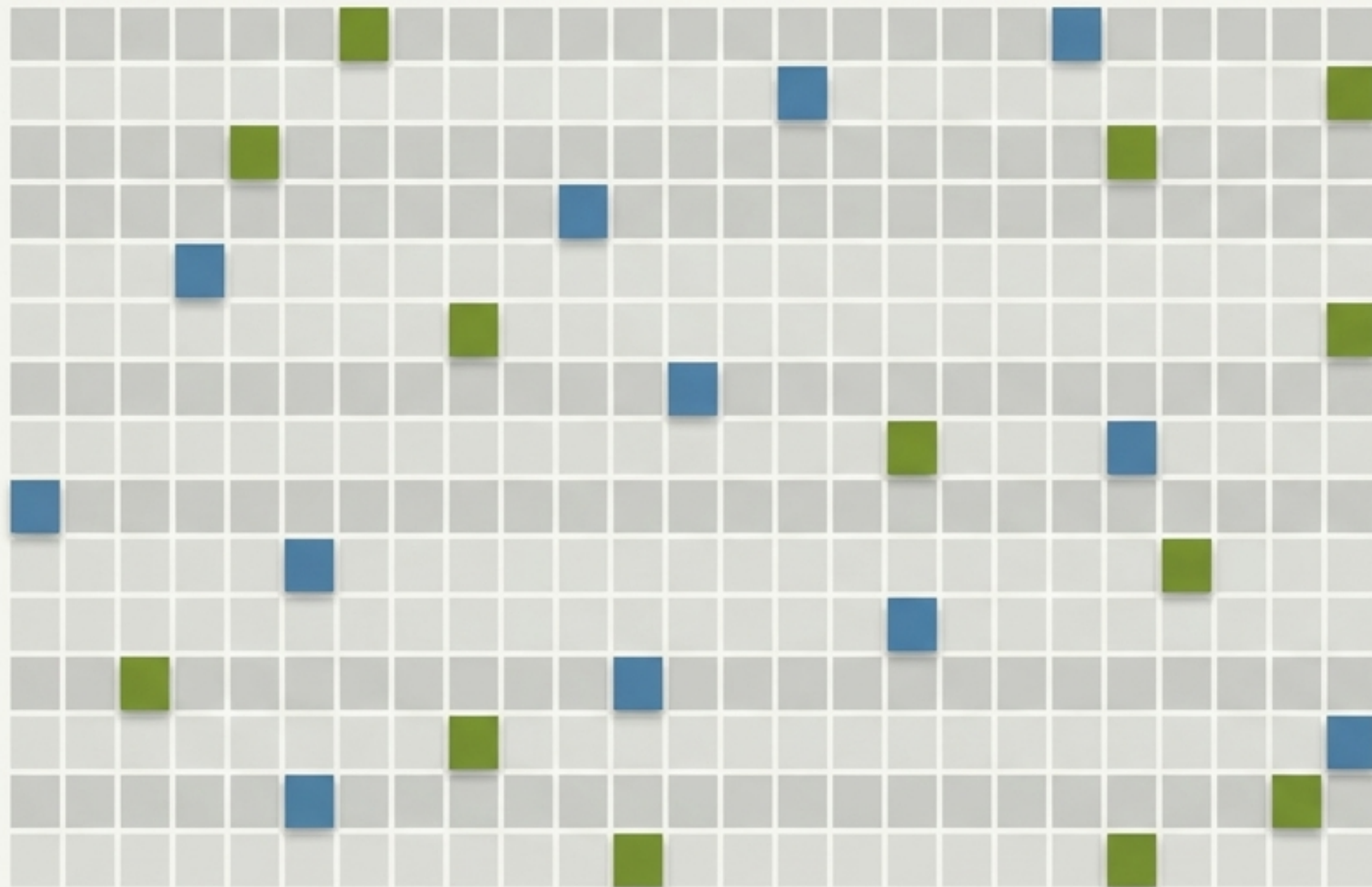
```
bootstrap=False
```

```
max_samples=1.0
```

```
max_features < 1.0
```

Method 4: Random Patches

Samples both rows and features simultaneously.

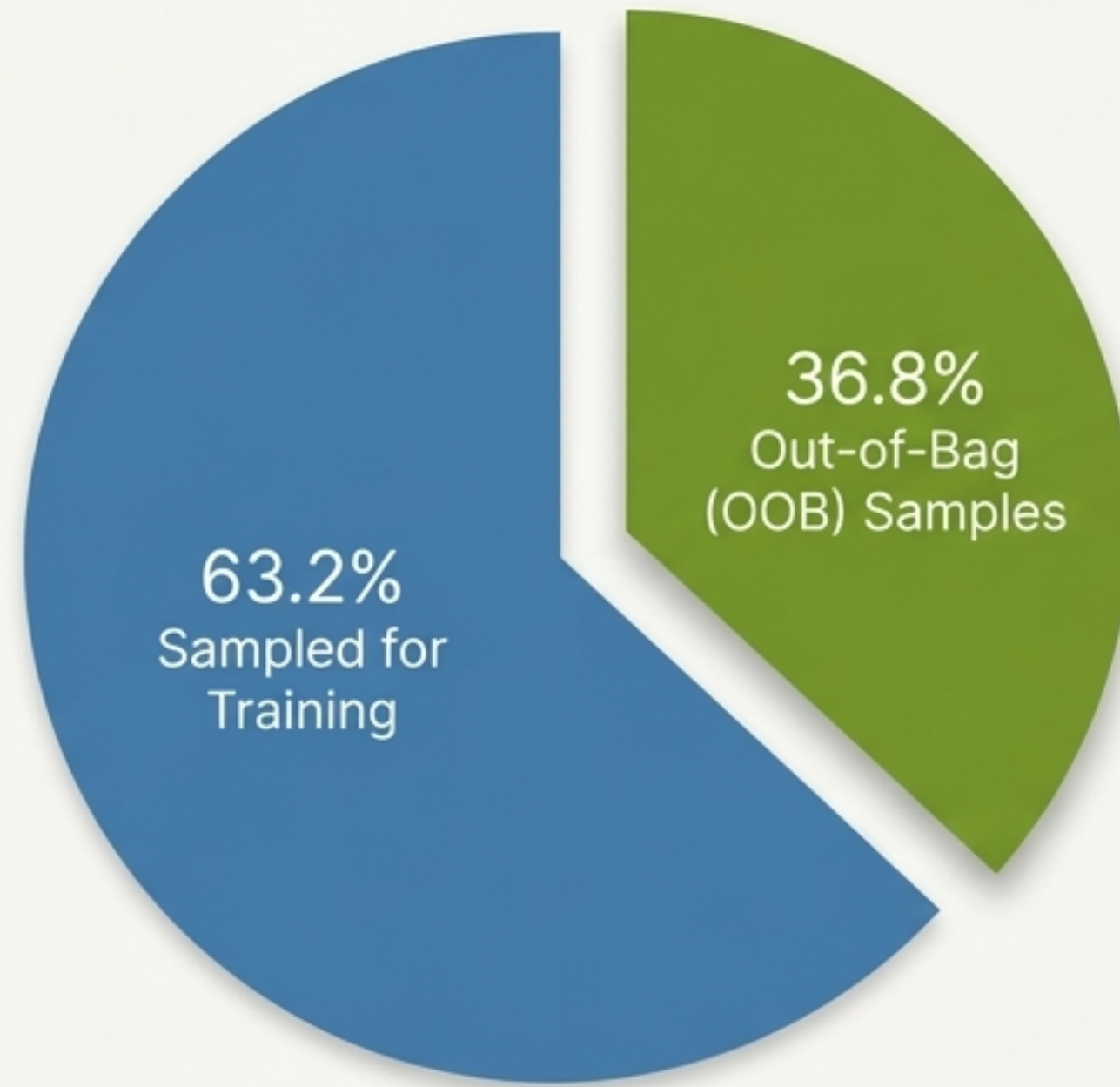


The ultimate high-dimensional tool. Ineffective on small, simple datasets, but powerful when tackling massive tables with thousands of columns.

```
max_samples < 1.0
```

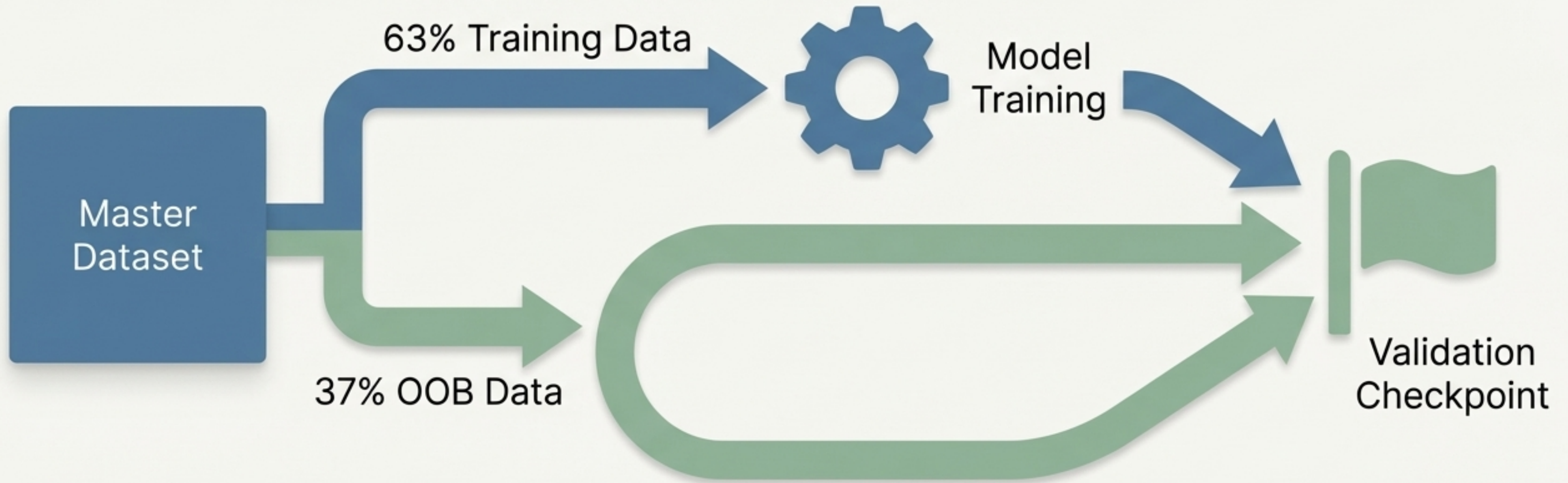
```
max_features < 1.0
```

The unseen thirty-seven percent



When sampling rows with replacement, statistically ~63% of the dataset is selected. The remaining ~37% of rows are never seen by the base models during training.

Free validation without data sacrifice



Instead of manually splitting data to create a validation set, use the unseen OOB samples to instantly verify model performance.

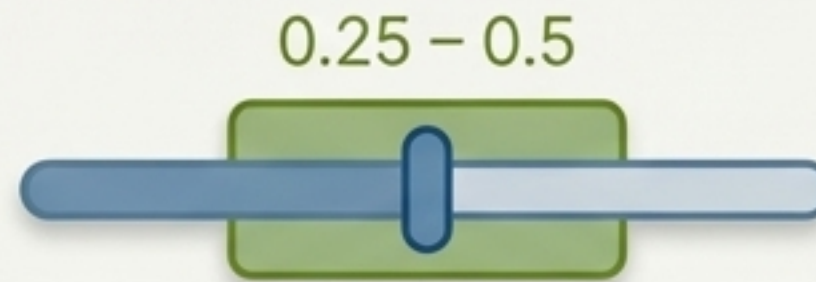
```
oob_score=True  
model.oob_score_
```

Empirical rules of thumb



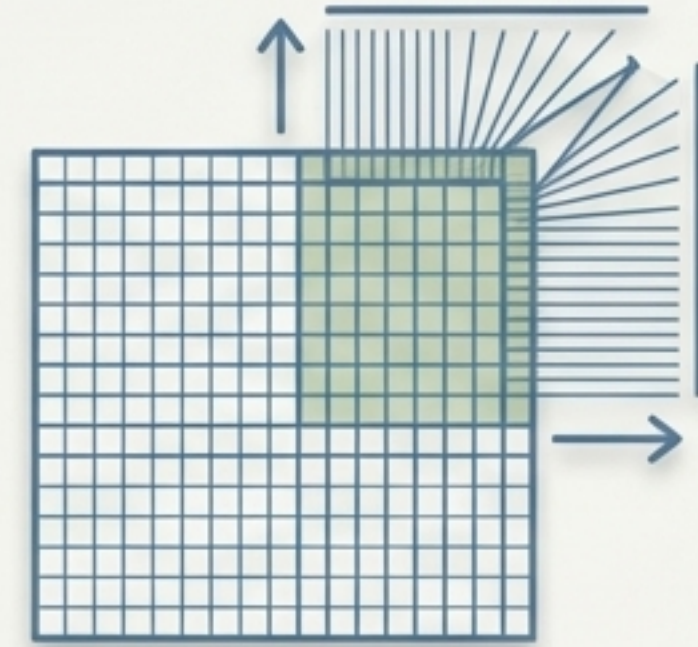
1. Bagging > Pasting

Bagging generally outperforms Pasting due to the variance introduced by duplicate rows.



2. The Sampling Sweet Spot

When defining row sampling limits, optimal results are consistently found setting `max_samples` between 0.25 and 0.5.

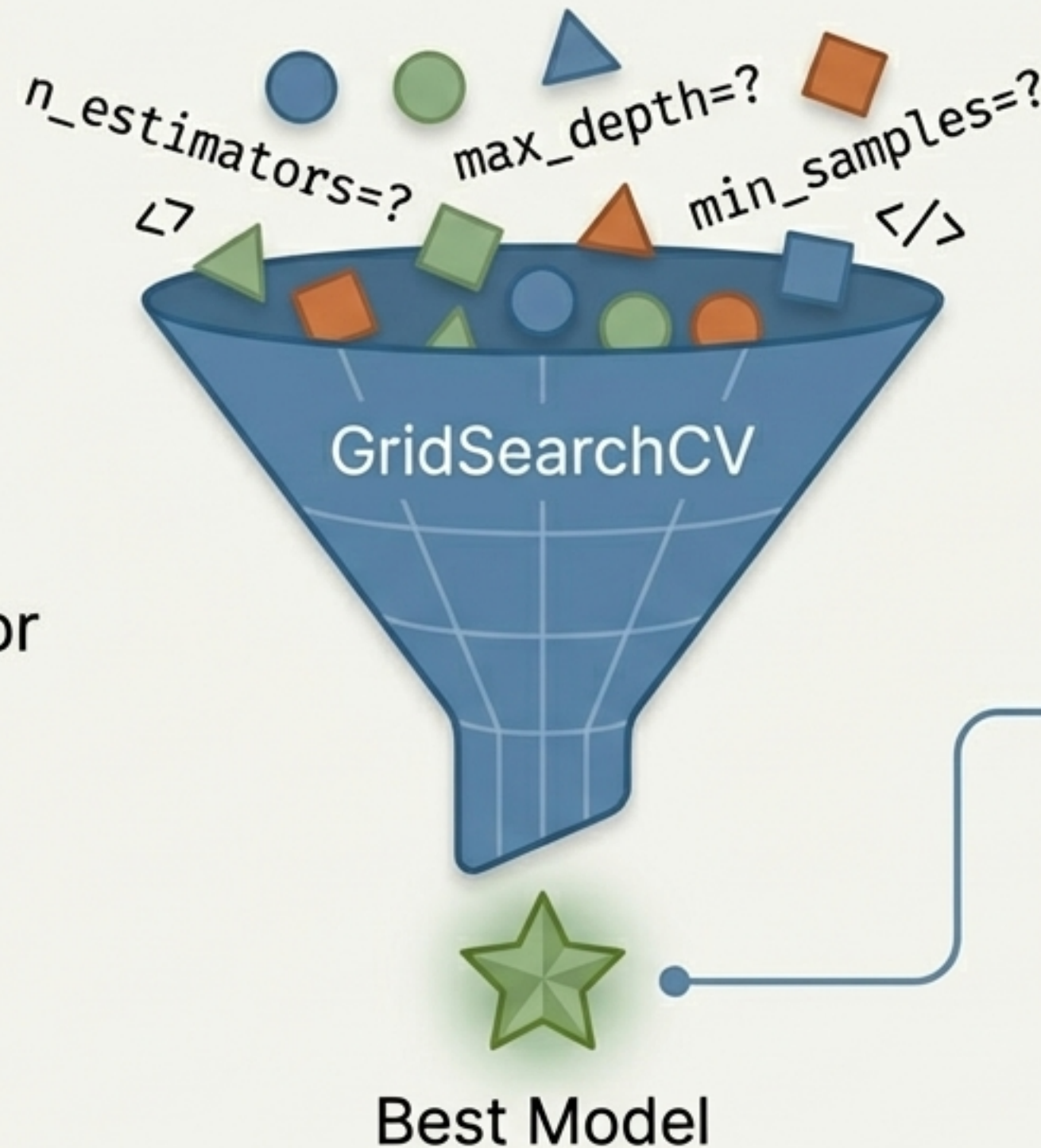


3. Dimensionality Dictates Method

Reserve Random Subspaces and Random Patches strictly for datasets with a massive volume of columns.

Stop guessing parameters

Use GridSearchCV to systematically test parameter combinations and identify the mathematical optimum for your specific dataset.



Example Winning Configuration:

`max_features=0.7`

`max_samples=0.5`

`n_estimators=500`

`bootstrap=True`

The Scikit-Learn Ensemble cheat sheet

Bagging

`bootstrap=True`

`max_samples < 1.0`

`max_features=1.0`

Pasting

`bootstrap=False`

`max_samples < 1.0`

`max_features=1.0`

Random Subspaces

`bootstrap=False`

`max_samples=1.0`

`max_features < 1.0`

Random Patches

`bootstrap=True/False`

`max_samples < 1.0`

`max_features < 1.0`

Keep `max_samples` between 0.25 and 0.5. Always leverage **`oob_score=True`** for free validation. Use Trees over SVMs.